EVOLUTIONARY COMPUTATION — EXERCISE 4

Candidate Moves in Steepest Local Search

Steepest LS (2-opt + node exchange) · Full neighborhood vs. candidate moves · Random starting solutions

Instances A and B · 200 nodes each · 100 selected per tour · 200 runs per method
Report generated 2026-10-29

Code at: github.com/mikolajdybizbanski/EC/blob/main/Lab4/main.cpp

Wiktor Bocheński · Mikołaj Dybizbański

1. Description of the problem

We are given two problem instances, **A** and **B**, each listing 200 nodes. Every node is described by three integers: its `x` and `y` coordinates on a plane, and a `cost`. Node indices are 0-based, taken from the row order in the instance file.

Goal. Select exactly 50% of the nodes (rounded up if the count is odd — for 200 nodes, exactly 100) and arrange the selected nodes into a **Hamiltonian cycle**: a closed path that visits every selected node exactly once and returns to its start. The objective to minimize is:

```
objective = total length of the cycle + sum of costs of the selected nodes
```

Distances between nodes are **Euclidean**, computed from the `(x, y)` coordinates and **rounded mathematically** to the nearest integer. The distance matrix is built once, immediately after reading an instance; from that point on every method only reads the distance matrix and the cost list.

A solution is a sequence of 100 distinct node indices, in visiting order; the cycle is implied by wrapping the last node back to the first.

2. Implemented methods

Both methods are **steepest local search** started from a **random solution** (100 random nodes in random order), using the neighborhood that performed best in the previous assignment:

- **Intra-route: two-edges exchange (2-opt)** — remove edges (a, a^+) and (b, b^+), add (a, b) and (a^+, b^+), i.e. reverse the path between them:

  $$\Delta = d[a][b] + d[a^+][b^+] - d[a][a^+] - d[b][b^+]$$

- **Inter-route: node exchange** — replace a selected node u (neighbors p, q) with an unselected node v:

  $$\Delta = cost[v] - cost[u] + d[p][v] + d[v][q] - d[p][u] - d[u][q]$$

In every iteration the move with the most negative Δ is applied; the search stops when no improving move exists. Deltas are computed in O(1); a position array gives the place of each node in the tour (−1 if it is not selected).

1. **Baseline — full neighborhood.** All n(n−3)/2 2-opt moves and all 100 × 100 exchanges are evaluated in every iteration.
2. **Candidate moves.** For every node `a` we precompute its **k = 10 nearest** nodes by `d[a][b] + cost[b]`; edge `(a, b)` with `b` in that list is a **candidate edge**. Only moves that **introduce at least one candidate edge** are generated: a loop over selected nodes `a`, then over the candidates `b` of `a`. Moves outside this set are never enumerated or checked. Each candidate edge can be introduced by exactly two moves:
   - `b` **selected** → two 2-opt moves: remove (a, a^+), (b, b^+) (adds (a, b), (a^+, b^+)), or remove (a^-, a), (b^-, b) (adds (a^-, b^-), (a, b));
   - `b` **not selected** → two exchanges: `b` replaces a^+ (adds (a, b)), or `b` replaces a^- (adds (b, a)). Exchanging `a` itself with `b` is *not* generated, as it would not introduce edge (a, b).

Both methods start from the **same 200 random solutions** per instance (fixed seed), so the results differ only because of the neighborhood. We also ran the candidate version for k ∈ {5, 15, 20} to choose k experimentally.

3. Pseudocode

3.1 Candidate lists

```
buildCandidateLists(distances, costs, k):
    for each node a:
        candidates[a] = the k nodes b != a with smallest distances[a][b] + costs[b]
    return candidates
```

3.2 Move evaluation (O(1))

```
considerTwoOpt(x, y, best):              // edges starting at tour positions x and y
    if x > y: swap(x, y)
    if y - x < 2 or (x == 0 and y == n-1): return      // edges share a node
    a, a+ = tour[x], tour[x+1];  b, b+ = tour[y], tour[(y+1) mod n]
    delta = d[a][b] + d[a+][b+] - d[a][a+] - d[b][b+]
    if delta < best.delta: best = (2opt, x, y, delta)

considerExchange(k, v, best):            // put unselected v at position k
    u = tour[k], p = tour[k-1 mod n], q = tour[k+1 mod n]
    delta = costs[v] - costs[u] + d[p][v] + d[v][q] - d[p][u] - d[u][q]
    if delta < best.delta: best = (exchange, k, v, delta)

apply(move):
    2opt:     reverse tour[x+1 .. y], update pos[] of reversed nodes
    exchange: pos[tour[k]] = -1;  tour[k] = v;  pos[v] = k
```

3.3 Steepest LS — full neighborhood (baseline)

```
steepestFull(tour):
    repeat:
        best = none (delta = 0)
        for x in 0..n-1, for y in x+2..n-1:  considerTwoOpt(x, y, best)
        for k in 0..n-1, for each unselected v:  considerExchange(k, v, best)
        if best is none: return tour        // local optimum
        apply(best)
```

3.4 Steepest LS — candidate moves

```
steepestCandidates(tour, candidates):
    repeat:
        best = none (delta = 0)
        for i in 0..n-1:
            a = tour[i]
            for b in candidates[a]:
                if b is selected:                   // j = pos[b]
                    considerTwoOpt(i, j, best)              // adds (a,b), (a+,b+)
                    considerTwoOpt(i-1 mod n, j-1 mod n, best) // adds (a-,b-), (a,b)
                else:
                    considerExchange(i+1 mod n, b, best)     // b replaces a+, adds (a,b)
                    considerExchange(i-1 mod n, b, best)     // b replaces a-, adds (b,a)
        if best is none: return tour
        apply(best)
```

3.5 Experiment driver

```
for instance in {A, B}:
    starts = 200 random solutions (fixed seed)
    for method in {full, candidates k=10, (k=5, 15, 20)}:
        for start in starts:
            tour = method(start);  record objective(tour) and time
        report min, max, average of objective and time; save the best tour
```

One iteration costs 14850 delta evaluations in the full neighborhood (4850 2-opt + 10 000 exchanges), but at most 2 · k · 100 = 2000 for k = 10: each of the k candidates of each of the 100 selected nodes triggers exactly two evaluations. In practice slightly fewer, because 2-opt pairs whose edges share a node are rejected.

4. Computational experiment

Each method was run 200 times per instance, from the same 200 random starting solutions. The candidate-moves method is highlighted; the best value in each column per instance is in bold.

Objective function

Instance	Method	Min	Max	Average
A	Steepest LS, full neighborhood (baseline)	**74116**	**80741**	**76255.5**
A	Steepest LS, candidate moves (k = 10) new	76522	85773	80262.2
B	Steepest LS, full neighborhood (baseline)	50607	55575	**52583.4**
B	Steepest LS, candidate moves (k = 10) new	**50306**	**55433**	52721.2

Running time of a single run [ms]

Instance	Method	Min	Max	Average	Speed-up
A	Steepest LS, full neighborhood (baseline)	6.266	14.657	8.214	1.00×
A	Steepest LS, candidate moves (k = 10) new	**1.020**	**3.992**	**1.282**	6.41×
B	Steepest LS, full neighborhood (baseline)	6.305	30.196	8.402	1.00×
B	Steepest LS, candidate moves (k = 10) new	**0.986**	**3.957**	**1.333**	6.30×

Speed-up = average time of the baseline / average time of the method. Compiled with g++ -O2, single thread. Candidate lists are computed once per instance (per k) before the runs and are excluded from the run time; for k = 10 this takes 0.60 ms on A and 0.55 ms on B, i.e. less than one local search run. Raw data: A_summary.csv, B_summary.csv.

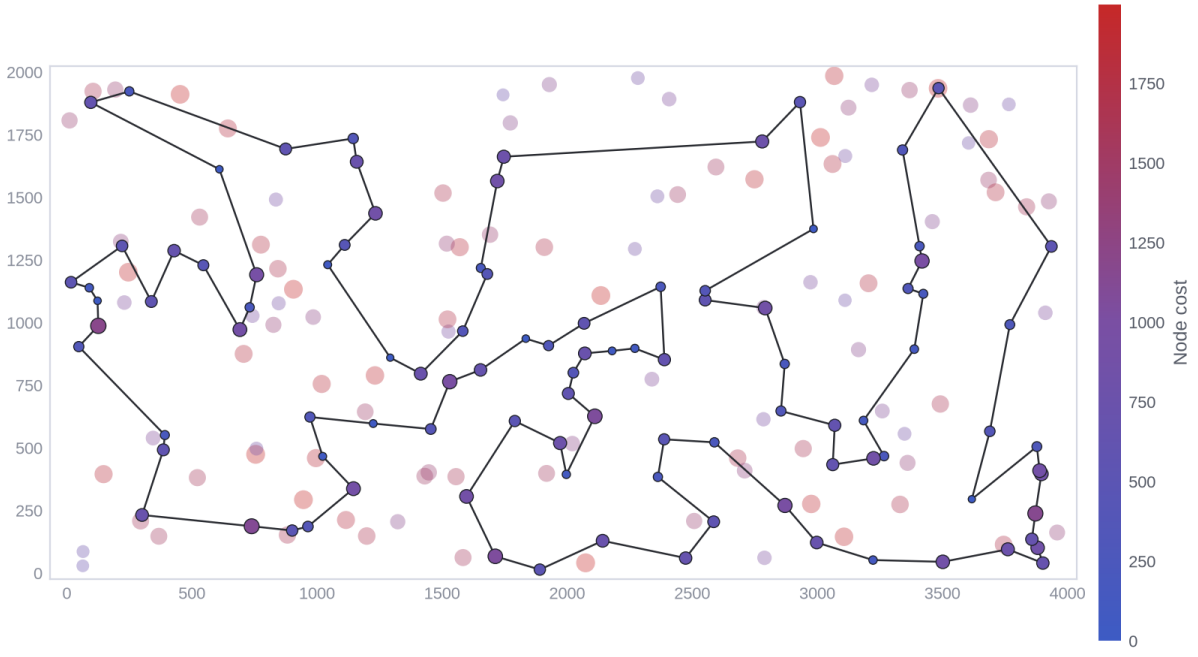
Choice of the number of candidates k

Instance	k	Min	Max	Average	Avg vs. full	Avg time [ms]	Speed-up
A	full	74116	80741	76255.5	+0.00%	8.214	1.00×
A	5	81339	93178	86422.8	+13.33%	0.839	9.79×
A	10	76522	85773	80262.2	+5.25%	1.282	6.41×
A	15	74583	81906	77615.2	+1.78%	2.094	3.92×
A	20	74124	80349	76452.6	+0.26%	2.710	3.03×
B	full	50607	55575	52583.4	+0.00%	8.402	1.00×
B	5	50976	58161	53973.3	+2.64%	0.710	11.83×
B	10	50306	55433	52721.2	+0.26%	1.333	6.30×
B	15	50525	55085	52507.7	-0.14%	2.077	4.05×
B	20	50278	54813	52568.1	-0.03%	2.741	3.07×

5. 2D visualizations of the best solution per instance/method

Node color and size encode cost (small/blue = cheap, large/red = expensive). Unselected nodes use the same color and size scale but are drawn faint and without an outline; the closed tour is drawn as a dark path.

Instance A

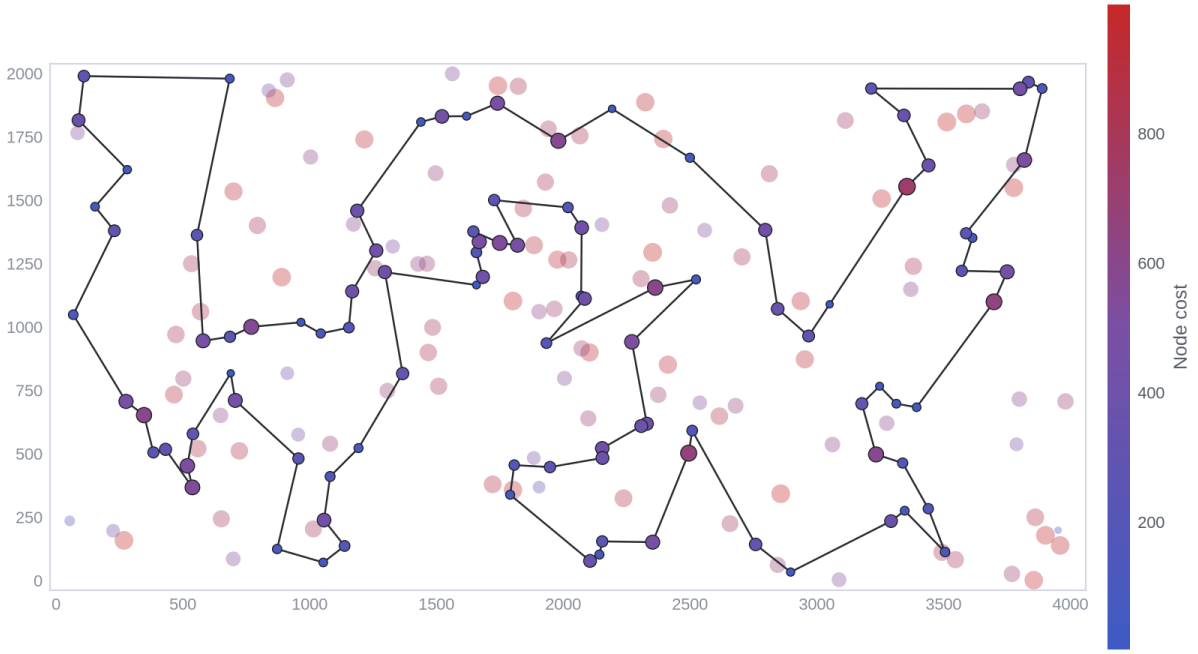


Steepest LS, full neighborhood (baseline) · objective 74116



Steepest LS, candidate moves (k = 10) · objective 76522

Instance B



Steepest LS, full neighborhood (baseline) · objective 50607



Steepest LS, candidate moves (k = 10) · objective 50306

6. Best solutions (node indices, 0-based)

Instance A, Steepest LS, full neighborhood (baseline) objective 74116

```
123, 160, 152, 108, 49, 60, 116, 149, 63, 119, 168, 97, 127, 69, 92, 171, 2, 189, 187, 30, 141, 87, 102, 185,
67, 144, 52, 33, 131, 36, 177, 179, 53, 140, 41, 135, 73, 45, 98, 25, 114, 181, 192, 158, 58, 84, 134, 138,
182, 57, 64, 88, 78, 193, 153, 75, 170, 146, 159, 107, 130, 47, 147, 29, 178, 1, 169, 3, 50, 93, 106, 89, 42,
151, 150, 176, 174, 8, 54, 9, 164, 136, 186, 112, 139, 51, 16, 35, 28, 180, 132, 76, 4, 71, 157, 91, 48, 31,
80, 100
```

Instance A, Steepest LS, candidate moves (k = 10) objective 76522

```
58, 158, 192, 181, 114, 135, 45, 87, 141, 30, 187, 189, 2, 171, 69, 127, 97, 83, 123, 100, 152, 108, 49, 11,
63, 168, 116, 149, 60, 82, 31, 48, 80, 91, 157, 71, 4, 76, 132, 162, 180, 28, 156, 172, 35, 185, 67, 144, 53,
179, 177, 36, 52, 190, 16, 51, 139, 112, 33, 131, 54, 9, 8, 164, 142, 174, 150, 106, 42, 89, 113, 93, 50, 3,
169, 1, 178, 184, 186, 136, 29, 55, 101, 147, 47, 130, 159, 170, 153, 78, 88, 107, 57, 182, 138, 151, 176, 72,
134, 84
```

Instance B, Steepest LS, full neighborhood (baseline) objective 50607

```
182, 80, 127, 129, 90, 143, 148, 10, 114, 196, 61, 105, 94, 81, 190, 70, 187, 18, 54, 192, 162, 104, 57, 89,
156, 48, 96, 198, 121, 130, 30, 38, 47, 140, 154, 26, 125, 177, 158, 116, 37, 146, 149, 28, 145, 65, 178, 103,
132, 29, 36, 197, 6, 170, 115, 71, 175, 134, 45, 157, 60, 40, 64, 21, 69, 160, 172, 136, 133, 8, 117, 120, 72,
46, 184, 3, 151, 135, 76, 27, 24, 34, 152, 181, 167, 59, 85, 82, 87, 179, 88, 31, 67, 99, 171, 74, 20, 2, 14,
153
```

Instance B, Steepest LS, candidate moves (k = 10) objective 50306

```
30, 183, 69, 160, 172, 136, 133, 8, 117, 135, 76, 162, 54, 192, 67, 24, 34, 11, 27, 3, 184, 151, 152, 181, 167,
59, 85, 51, 82, 87, 179, 88, 99, 39, 171, 17, 188, 2, 110, 14, 20, 74, 153, 182, 52, 90, 129, 80, 127, 78, 10,
114, 196, 61, 105, 94, 81, 187, 70, 190, 57, 89, 72, 120, 96, 198, 121, 147, 116, 37, 146, 149, 28, 145, 65,
158, 177, 125, 170, 115, 6, 29, 132, 197, 36, 62, 175, 134, 45, 157, 60, 40, 71, 155, 97, 154, 140, 47, 38, 21
```

7. Solution checker

All 4 best solutions above were checked with `checker.py`, which independently re-reads the instance data, counts the visited nodes and recomputes the objective (cycle length + node costs) from scratch. We additionally verified that no index repeats. In addition, `main.cpp` runs a self-check (`LocalSearch::verifyDeltas`) before the experiment: on 5 random solutions per instance it applies every 2-opt and exchange move and compares its delta with a full re-evaluation of the resulting tour — A: 74250 moves, 0 mismatches; B: 74250 moves, 0 mismatches.

Instance	Method	Nodes	main.cpp	checker.py	Status
A	Steepest LS, full neighborhood (baseline)	100	74116	74116	✓ valid
A	Steepest LS, candidate moves (k = 10)	100	76522	76522	✓ valid
B	Steepest LS, full neighborhood (baseline)	100	50607	50607	✓ valid
B	Steepest LS, candidate moves (k = 10)	100	50306	50306	✓ valid

ALL 4 VALID — the recomputed objective matched the value reported by `main.cpp` exactly in every case.

8. Conclusions

- **Candidate moves make steepest local search much faster.** With k = 10 the average run takes 1.28 ms on A and 1.33 ms on B, against 8.21 / 8.40 ms for the full neighborhood — a **6.4× / 6.3× speed-up**. Each iteration evaluates at most 2000 instead of 14 850 moves (a 7.4× ratio).
- **The quality cost depends on the instance.** On B, k = 10 is practically as good as the full neighborhood (average +0.26%, and it even found a better best solution: 50306 vs. 50607). On A the loss is larger (average +5.25%, best 76522 vs. 74116). Candidate moves exclude improving moves that need a long edge, for example joining two clusters or reaching a cheap node that is far away. A likely reason for the difference is that node costs on A are about twice as large as on B, which changes which nodes end up among the 10 nearest.
- **k controls the trade-off.** k = 5 is the fastest but clearly worse (+13.33% on A, +2.64% on B). With k = 20 the averages are within 0.26% (A) and 0.03% (B) of the full neighborhood, while the method is still 3.0× / 3.1× faster. With k = 10 we get a bigger speed-up and lose some quality on A; k ≈ 15–20 keeps the quality of the baseline.
- Candidate edges use `d + cost`, so they favor cheap nodes. The best tours of both methods look very similar in the plots and avoid most of the large, red (expensive) nodes.
- Run times were measured on a shared laptop and are only indicative; the ratios between the methods matter more than the absolute values.